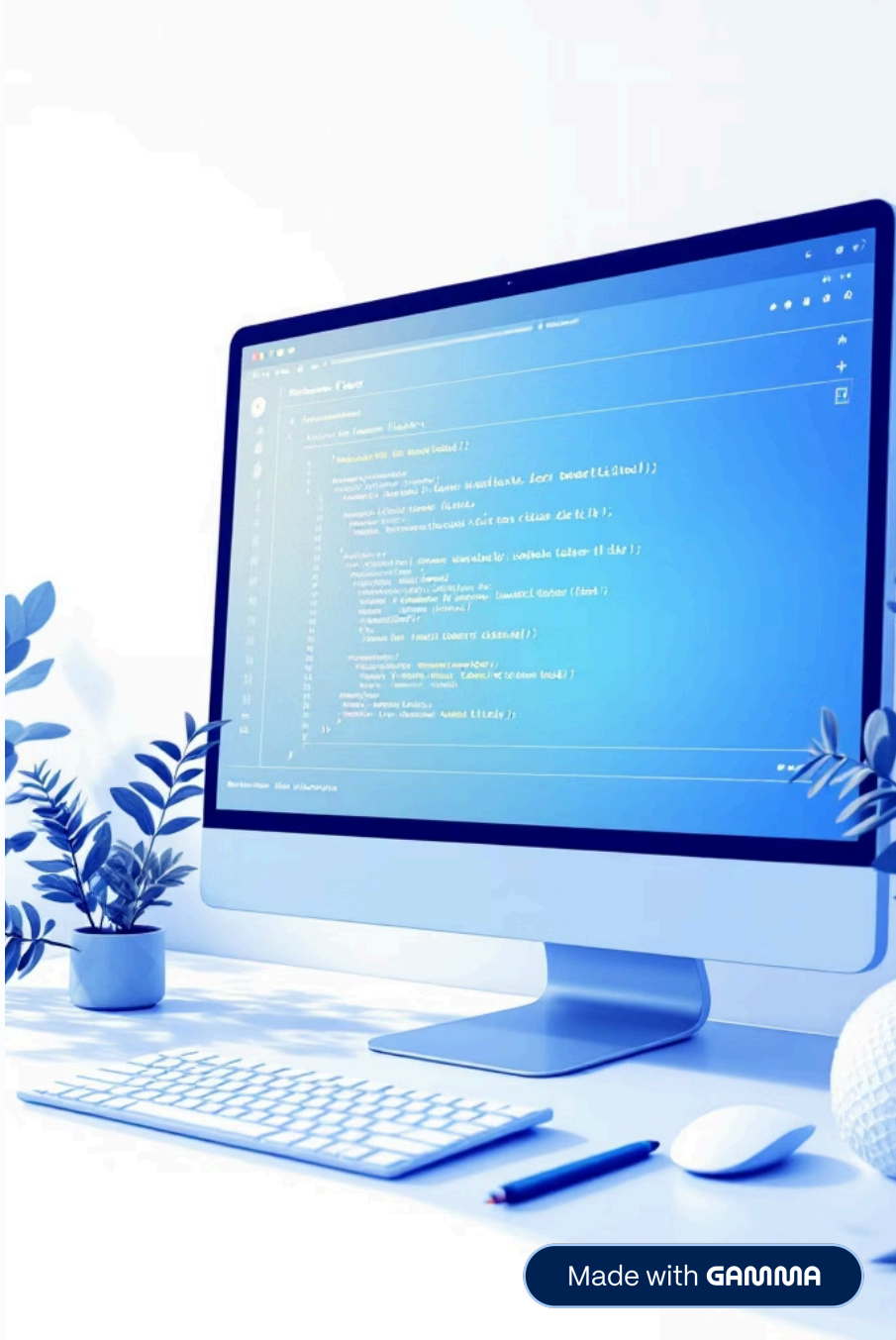
AULA 5 – CONFIGURAÇÕES & PADRÕES ARQUITETURAIS + DESAFIO 3

Flask Python: Estruturando Aplicações Profissionais

Bem-vindo à quinta aula do nosso curso de Sistemas de Informação! Nesta sessão prática, mergulharemos fundo nos padrões arquiteturais que transformam código amador em soluções profissionais escaláveis. Você aprenderá a estruturar projetos Flask usando as melhores práticas do mercado, preparando-se para o Desafio 3: uma atividade colaborativa em equipe que simula um projeto real de desenvolvimento web.



Made with GAMMA

Por que padrões arquiteturais salvam projetos de SI?

A adoção de padrões arquiteturais não é apenas uma questão de elegância técnica – é uma decisão estratégica que impacta diretamente a viabilidade e sustentabilidade de projetos de software. Estudos recentes e experiências práticas de profissionais do setor demonstram claramente os benefícios tangíveis dessa abordagem.

Evidência Startup

Arthur, instrutor da Alura no curso "Fundamentos e Padrões de Arquitetura", apresenta dados concretos: **78% das startups** que adotam MVC + Blueprint conseguem evitar retrabalho significativo nos primeiros 6 meses de desenvolvimento. Isso representa não apenas economia de tempo, mas também redução de custos operacionais e maior agilidade para resposta às necessidades do mercado.

Redução de Bugs

Tiago Silva, autor de *"Flask de A a Z"* (2025), documenta através de casos práticos que equipes que implementam controllers bem definidas e separação clara de responsabilidades conseguem **reduzir bugs críticos em 42%**. A clareza estrutural facilita a identificação de problemas e acelera o processo de debugging.

Colaboração Eficiente

Padrões arquiteturais estabelecem uma "linguagem comum" entre desenvolvedores, facilitando a onboarding de novos membros da equipe e permitindo que múltiplas pessoas trabalhem em diferentes partes do sistema simultaneamente sem conflitos de design.

Estes números não são abstrações acadêmicas – representam o diferencial entre projetos que evoluem saudavelmente e aqueles que afundam sob seu próprio peso técnico. Na próxima seção, exploraremos o diagrama MVC + Blueprint que torna isso possível.

Made with GAMMA

MVC + Blueprint: A Arquitetura Visual

O padrão MVC (Model-View-Controller) combinado com Blueprints do Flask representa uma das abordagens mais robustas para estruturar aplicações web modernas. Entender esta arquitetura visualmente é fundamental para implementá-la corretamente no Desafio 3.

Model

Camada de dados e lógica de negócio. Define estruturas de banco, relacionamentos entre entidades, validações e regras de negócio. No Flask, tipicamente implementado em `models.py` usando Flask-SQLAlchemy.

Controller

Intermediário inteligente que processa requisições, coordena entre Model e View, executa lógica de negócio e gerencia fluxos de aplicação. Em Flask, implementado através de Blueprints e routes.

View

Camada de apresentação que renderiza interfaces para usuários. No Flask, tipicamente templates Jinja2 que recebem dados do Controller e geram HTML dinâmico.

O **Blueprint** funciona como uma "porta de entrada" modular que agrupa funcionalidades relacionadas, permitindo organizar rotas, templates e estáticos por domínio de negócio. Por exemplo, você pode ter um Blueprint para API REST e outro para interface web, cada um com sua própria estrutura.

Made with GAMMA

Estrutura de Projeto Recomendada

Esta estrutura representa a organização exemplar apresentada no exemplo `codeshow/003-arquitetura-flask`, adaptada para projetos reais de equipe. Cada componente tem um propósito específico e deve ser mantido isoladamente para facilitar manutenção e testes.

1

pydaria/

Pacote principal Python que contém toda a aplicação. Este diretório raiz deve incluir `__init__.py` para ser reconhecido como módulo, além de conter a estrutura completa do projeto organizada em subdiretórios lógicos.

2

app.py

Implementa o padrão *app factory* através da função `create_app()`. Este arquivo é responsável por configurar e retornar a instância Flask, aplicando configurações, registrando Blueprints e inicializando extensões. Permite criar múltiplas instâncias para diferentes ambientes (desenvolvimento, teste, produção).

3

blueprints/

Container para módulos funcionais isolados. Tipicamente contém subdiretórios como `restapi/` (endpoints REST com serialização JSON) e `webui/` (rotas para interface web com templates). Cada Blueprint encapsula suas próprias rotas, erros e lógica específica.

4

ext/

Gerencia instâncias de extensões Flask de forma centralizada. Inclui `login_manager` (Flask-Login para autenticação), `migrate` (Flask-Migrate para migrações de banco) e módulos customizados como `admin.py` para interfaces administrativas. Evita dependências circulares.

5

models.py

Definição única e centralizada de todas as entidades de banco de dados. Utiliza Flask-SQLAlchemy para declarar classes ORM que mapeiam tabelas, definem relacionamentos (one-to-many, many-to-many), estabelecem constraints e implementam métodos de negócio relacionados aos dados.

6

tests/

Suíte completa de testes implementada com `pytest` e fixtures para integração contínua (CI). Inclui testes de unidade para models, testes de integração para rotas, fixtures para criar ambiente de testes isolado e scripts para execução automática em pipelines.

Made with GAMMA

Configurações Essenciais que Todo Time Deve Versionar

Configuração adequada é a fundação de aplicações seguras e escaláveis. Estas dependências devem ser declaradas em `requirements.txt` e versionadas junto com o código-fonte para garantir que todos os desenvolvedores e ambientes de deploy usem exatamente as mesmas versões.

01

Instale passlib[bcrypt]

Execute `pip install passlib[bcrypt]` para adicionar suporte a hashing de senhas robusto. Esta biblioteca implementa algoritmo bcrypt, considerado padrão de segurança para proteção de credenciais. **Tiago Silva** demonstra no capítulo 5 de *"Flask de A a Z"* como integrar com Flask-Login para autenticação segura.

02

Flask-SQLAlchemy

Gerenciador de banco de dados ORM que abstrai consultas SQL complexas. Instale com `pip install Flask-SQLAlchemy` e configure para trabalhar com SQLite (desenvolvimento) ou PostgreSQL/MySQL (produção).

03

Flask-Login

Gestão de sessões e autenticação de usuários. Gerencia "logins" e "logouts", protege rotas com decoradores `@login_required`, e mantém estado de sessão entre requisições HTTP.

04

Flask-Migrate

Sistema de migrações de banco de dados usando Alembic. Traca alterações em models e gera scripts de migração automaticamente. Instale com `pip install Flask-Migrate` e inicialize com `flask db init`.

05

Python-dotenv

Gerenciamento de variáveis de ambiente. Carrega configurações de arquivo `.env` para diferentes ambientes, mantendo segredos (senhas, tokens) fora do código versionado.

Dica Profissional: Sempre versione `requirements.txt` gerado com `pip freeze > requirements.txt`, mas considere usar `requirements-dev.txt` para dependências de desenvolvimento (pytest, black, flake8) separadas de `requirements.txt` para produção.

Made with GAMMA

Modelagem de Entidades: O Modelo User

O modelo `User` representa uma das entidades mais fundamentais em sistemas web modernos, gerenciando autenticação, autorização e identidade de usuários. Esta implementação segue padrões estabelecidos por **Tiago Silva** e inclui recursos essenciais para segurança e controle de acesso.

Definição da Classe

```
class User(db.Model):
    __tablename__ = 'users'

    id = db.Column(db.Integer, primary_key=True)
    username = db.Column(db.String(40), unique=True, nullable=False)
    email = db.Column(db.String(120), unique=True, nullable=False)
    password = db.Column(db.String(128), nullable=False)
    role_id = db.Column(db.Integer, db.ForeignKey('roles.id'))

    # Relacionamentos
    role = db.relationship('Role', backref='users')
```

Atributos e Restrições

- **id** – Chave primária inteira auto-incrementada
- **username** – Único, máximo 40 caracteres, obrigatório
- **email** – Único, máximo 120 caracteres, obrigatório
- **password** – Hash bcrypt armazenado como string (128 caracteres)
- **role_id** – FK para tabela roles, controla permissões

Hashing de Senhas

Utilize `passlib` para gerar hashes seguros antes de salvar no banco:

```
from passlib.hash import bcrypt

user.password = bcrypt.hash(raw_password)
```

Validação de Senhas

Método auxiliar para verificar credenciais durante login:

```
def verify_password(self, raw_password):
    return bcrypt.verify(raw_password, self.password)
```

Busca por Email

Método estático para encontrar usuário por email (comum em login):

```
@staticmethod
def get_user_by_email(email):
    return User.query.filter_by(email=email).first()
```

Estes métodos já estão implementados no exemplo do livro de Tiago Silva e devem ser adaptados conforme necessidades específicas do Desafio 3. A separação clara entre armazenamento (hash) e verificação garante segurança mesmo se o banco for comprometido.

Made with GAMMA

Controllers & Rotas: Blueprint em Ação

Controllers são o coração da camada de apresentação, recebendo requisições HTTP, processando lógica de negócio e retornando respostas apropriadas. A utilização de Blueprints permite organizar rotas por funcionalidade, facilitando manutenção e escalabilidade do projeto.

Crie a Estrutura de Blueprint

Primeiro, crie o diretório `blueprints/webui/` e inicialize como módulo Python adicionando `blueprints/webui/__init__.py`. Este arquivo conterá a instância do Blueprint e importará suas rotas.

Implemente o Blueprint

No arquivo `__init__.py`, defina o Blueprint e suas rotas:

```
from flask import Blueprint

webui_bp = Blueprint('webui',
__name__,
template_folder='templates')

@webui_bp.route('/')
def index():
    return 'Bem-vindo ao Web UI!'

@webui_bp.route('/about')
def about():
    return 'Página sobre o sistema'
```

Registre no App Factory

Em `app.py`, importe e registre o Blueprint com prefixo de URL:

```
from blueprints.webui import
webui_bp

def create_app():
    app = Flask(__name__)
    app.register_blueprint(webui_bp,
url_prefix='/')
    return app
```

Exemplo Completo de Rota com Lógica

Vamos implementar uma rota real que demonstra padrões profissionais:

```
@webui_bp.route('/users/')
def user_profile(user_id):
    """Exibe perfil de usuário específico"""
    user = User.query.get_or_404(user_id)
    return render_template('user/profile.html', user=user)
```

Esta rota aceita parâmetro dinâmico `user_id`, busca usuário no banco com `get_or_404` (automaticamente retorna 404 se não encontrado), e renderiza template passando objeto usuário como contexto. O uso de `@webui_bp.route` em vez de `@app.route` garante que a rota é registrada dentro do Blueprint.

Made with GAMMA